

ASSIGNMENT

Question: -1

Create a NumPy array of 50 random numbers and find the 2nd largest and 3rd smallest element.

Ans: - Code & Output

```
import numpy as np
arr = np.random.randint(1, 100, 50)
print("Array is given:")
print(arr)
sorted_arr = np.sort(arr)
second_largest = sorted_arr[-2]
third_smallest = sorted_arr[2]
print("\n2nd Largest Element:", second_largest)
print("3rd Smallest Element:", third_smallest)
```

✓ 0.0s

Array is given:

```
[ 7 16 96 26 52 81 67 10 54 23 90 91 65 51  4 84 52 31  7 16 45 82 69 50
 15 47 30 62 89 89 91 24 64 89 95 58 18 46 66 12  5 26 28 55 79 88 89 56
 90 37]
```

2nd Largest Element: 95
3rd Smallest Element: 7

Question: - 2

Given a Data-Frame, identify all columns having more than 20% missing values and remove them.

Ans: Code & Output

```
df = pd.read_csv(r"D:\Data Science with AI & ML\SITP-12th-May\PANDAS\loan_approval.csv")
missing_percent = df.isnull().mean() * 100
print("Missing Percentage in Each Column:")
print(missing_percent)
cols_to_remove = missing_percent[missing_percent > 20].index
print("\nColumns to Remove (>20% missing values):")
print(list(cols_to_remove))
df_cleaned = df.drop(columns=cols_to_remove)
print("\nRemaining Columns:")
print(df_cleaned.columns.tolist())
```

```

Missing Percentage in Each Column:
name          0.0
city          0.0
income        0.0
credit_score   0.0
loan_amount    0.0
years_employed 0.0
points        0.0
loan_approved  0.0
dtype: float64

Columns to Remove (>20% missing values):
[]

Remaining Columns:
['name', 'city', 'income', 'credit_score', 'loan_amount', 'years_employed', 'points', 'loan_approved']

```

Question: - 3

Create a NumPy matrix and find its transpose, determinant, rank, and inverse.

Ans: - Code and Output

```

import numpy as np
A = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]])
print("Matrix of A:")
print(A)
transpose_A = A.T
print("\nTranspose of A:")
print(transpose_A)
det_A = np.linalg.det(A)
print("\nDeterminant of A:")
print(det_A)
rank_A = np.linalg.matrix_rank(A)
print("\nRank of A:")
print(rank_A)
if det_A != 0:
    inverse_A = np.linalg.inv(A)
    print("\nInverse of A:")
    print(inverse_A)
else:
    print("\nInverse does not exist (determinant is zero).")

```

```
Matrix of A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Transpose of A:
[[1 4 7]
 [2 5 8]
 [3 6 9]]

Determinant of A:
0.0

Rank of A:
2

Inverse does not exist (determinant is zero).
```

Question: - 4

Using Pandas, find the top 5 categories with the highest average value of a numerical column.

Ans: Code & Output

```
df = pd.read_csv(r"D:\Data Science with AI & ML\SITP-12th-May\PANDAS\loan_approval.csv")
top_5 = (
    df.groupby("city")["income"]
      .mean()
      .sort_values(ascending=False)
      .head(5)
)
print("Top 5 Cities by Average Income:")
print(top_5)
```

✓ 0.0s

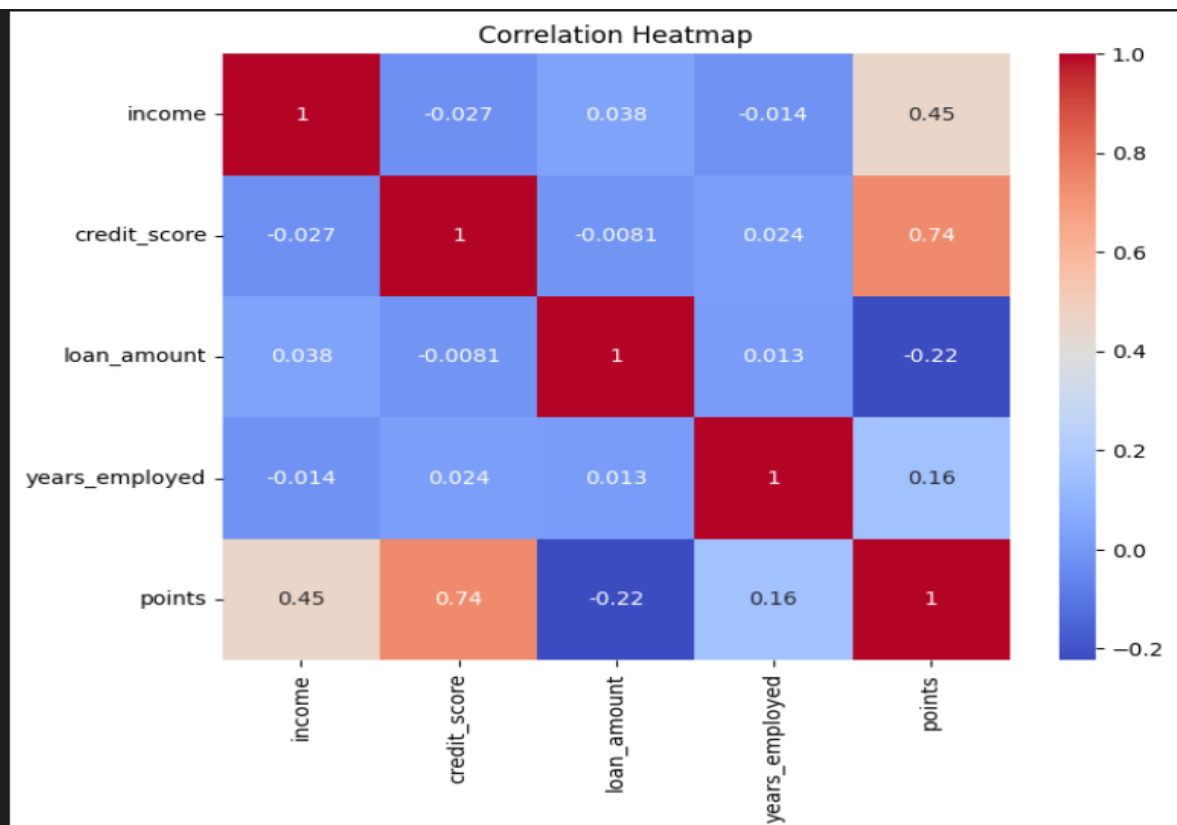
```
Top 5 Cities by Average Income:
city
New Nicolestad      149964.0
Michellemouth      149871.0
Martinezview       149806.0
Heatherview        149732.0
West Melissamouth  149725.0
Name: income, dtype: float64
```

Question: - 5

Create a Seaborn heatmap and identify the three most highly correlated feature pairs.

Ans: Code & Output

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
df = pd.read_csv(r"D:\Data Science with AI & ML\SITP-12th-May\PANDAS\loan_approval.csv")
numeric_df = df.select_dtypes(include=np.number)
corr_matrix = numeric_df.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
corr_pairs = corr_matrix.unstack()
corr_pairs = corr_pairs[corr_pairs != 1]
corr_pairs = corr_pairs.sort_values(ascending=False)
corr_pairs = corr_pairs.drop_duplicates()
print("Top 3 Highly Correlated Feature Pairs:")
print(corr_pairs.head(3))
```



```
Top 3 Highly Correlated Feature Pairs:
points    credit_score    0.740899
income    points          0.451142
points    years_employed  0.156441
dtype: float64
```

Question: - 6

Generate a dataset with NumPy and visualize its distribution using Histogram, KDE Plot, and Box Plot.

Ans: Code & Output

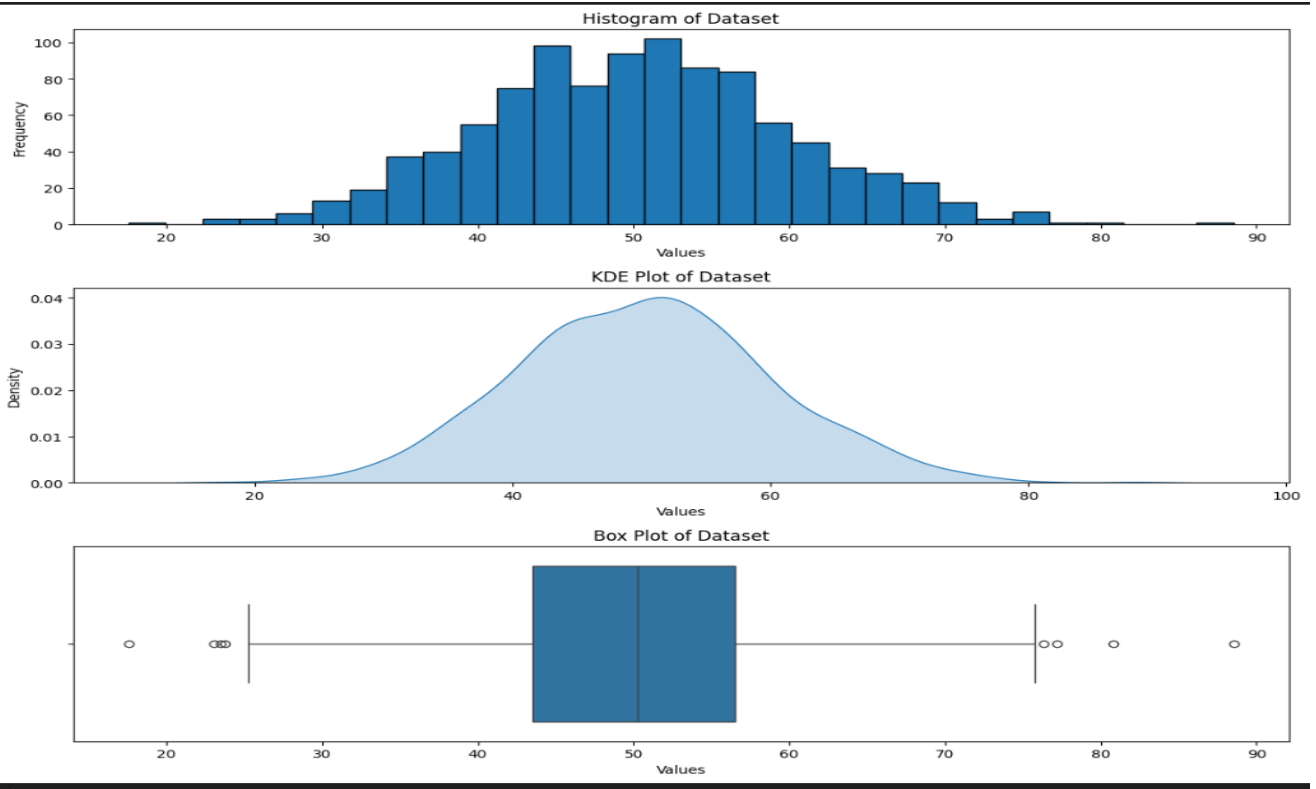
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
data = np.random.normal(loc=50, scale=10, size=1000)
plt.figure(figsize=(12, 10))

# Histogram
plt.subplot(3, 1, 1)
plt.hist(data, bins=30, edgecolor='black')
plt.title('Histogram of Dataset')
plt.xlabel('Values')
plt.ylabel('Frequency')

# KDE Plot
plt.subplot(3, 1, 2)
sns.kdeplot(data, fill=True)
plt.title('KDE Plot of Dataset')
plt.xlabel('Values')
plt.ylabel('Density')

# Box Plot
plt.subplot(3, 1, 3)
sns.boxplot(x=data)
plt.title('Box Plot of Dataset')
plt.xlabel('Values')

plt.tight_layout()
plt.show()
```



Question: - 7

Using Pandas GroupBy, find the highest, lowest, and average sales for each category.

Ans: Code & Output

```
import pandas as pd

df = pd.read_csv(r"D:\Data Science with AI & ML\SITP-12th-May\PANDAS\sales.csv")

result = df.groupby('Item Type')['Total Revenue'].agg(
    Highest_Sales='max',
    Lowest_Sales='min',
    Average_Sales='mean'
)

print(result)
```

✓ 1.4s

Item Type	Highest_Sales	Lowest_Sales	Average_Sales
Baby Food	2533654.00	324971.44	1.478618e+06
Beverages	445508.05	221117.00	3.363493e+05
Cereal	1780539.20	140287.40	7.604141e+05
Clothes	1082418.40	97040.64	5.990225e+05
Cosmetics	4324782.40	745426.00	2.815501e+06
Fruits	89623.98	4870.26	4.664813e+04
Household	5997054.98	188452.14	3.321079e+06
Meat	2492526.12	2011149.63	2.251838e+06
Office Supplies	5396577.27	617347.08	2.548782e+06
Personal Care	707454.88	22312.29	3.980905e+05
Snacks	1117953.66	339490.50	6.935778e+05
Vegetables	1245112.92	19103.44	5.148428e+05

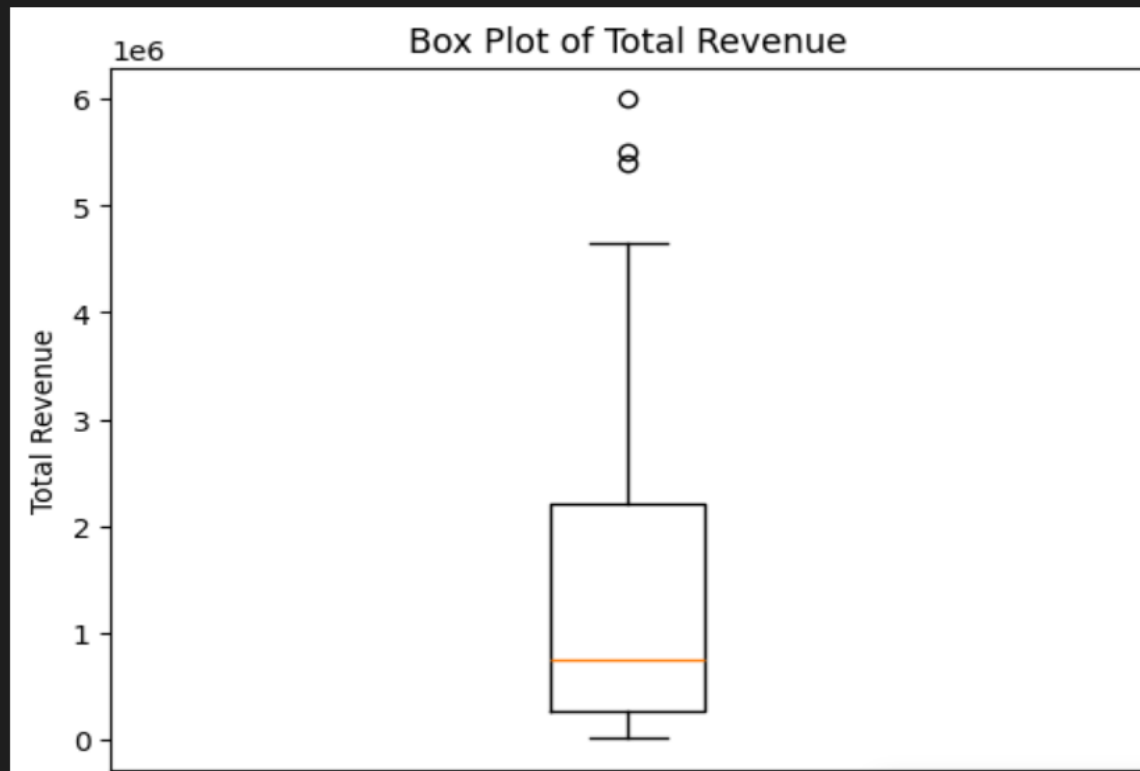
Question: - 8

Detect outliers in a numerical column using the IQR method and visualize them using a Box Plot.

Ans: Code & Output

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv(r"D:\Data Science with AI & ML\SITP-12th-May\PANDAS\sales.csv")
col = "Total Revenue"
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Number of Outliers:", len(outliers))
plt.boxplot(df[col])
plt.title("Box Plot of Total Revenue")
plt.ylabel("Total Revenue")
plt.show()
```

```
Lower Bound: -2646263.9925
Upper Bound: 5127029.8875
Number of Outliers: 3
```



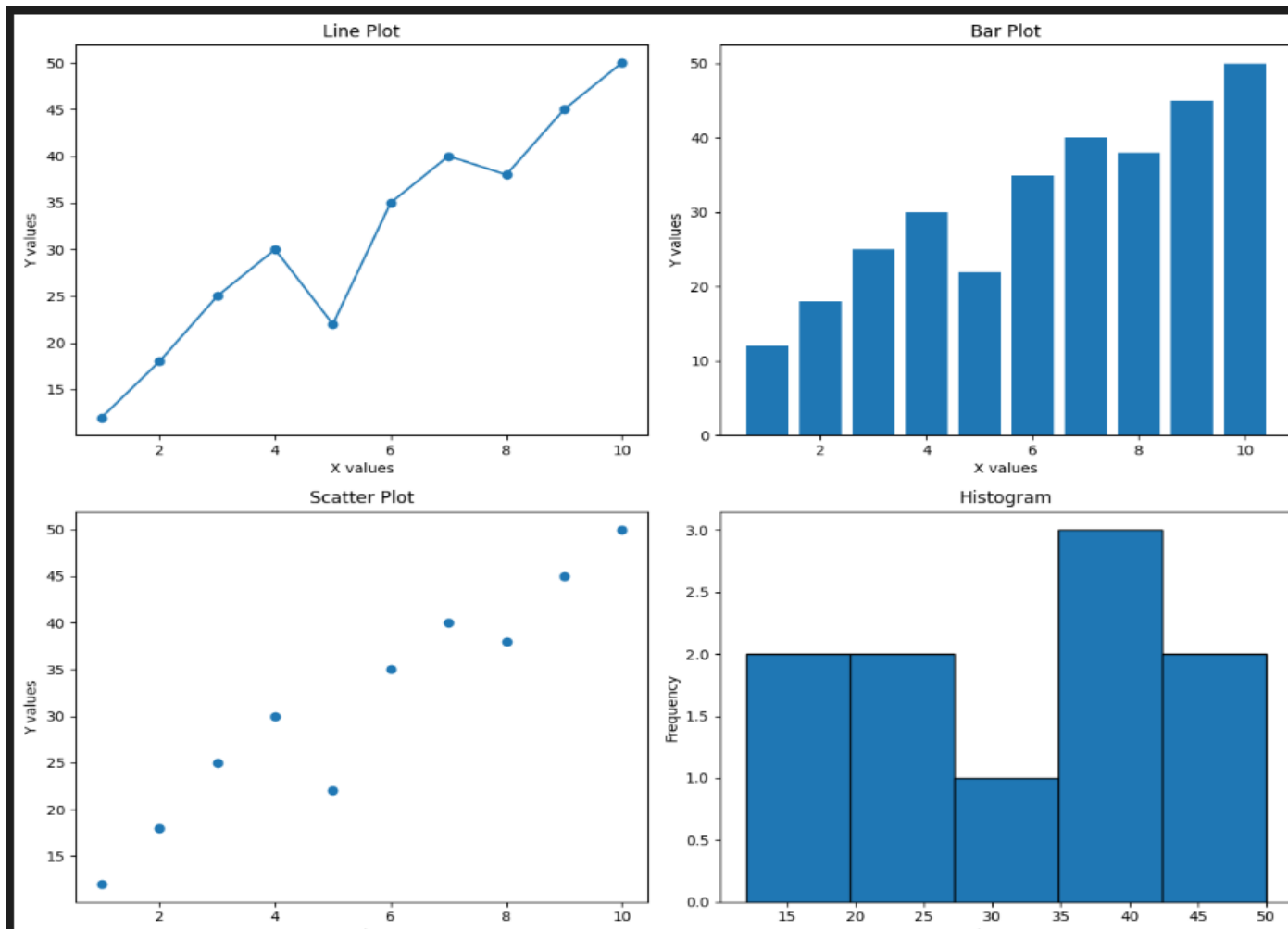
Question: - 9

Create a Matplotlib figure containing 4 subplots:

Line Plot, Bar Plot, Scatter Plot, Histogram

Ans: Code & Output

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(1, 11)
y = np.array([12, 18, 25, 30, 22, 35, 40, 38, 45, 50])
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
# Line Plot
axes[0, 0].plot(x, y, marker='o')
axes[0, 0].set_title("Line Plot")
axes[0, 0].set_xlabel("X values")
axes[0, 0].set_ylabel("Y values")
# Bar Plot
axes[0, 1].bar(x, y)
axes[0, 1].set_title("Bar Plot")
axes[0, 1].set_xlabel("X values")
axes[0, 1].set_ylabel("Y values")
# Scatter Plot
axes[1, 0].scatter(x, y)
axes[1, 0].set_title("Scatter Plot")
axes[1, 0].set_xlabel("X values")
axes[1, 0].set_ylabel("Y values")
# Histogram
axes[1, 1].hist(y, bins=5, edgecolor='black')
axes[1, 1].set_title("Histogram")
axes[1, 1].set_xlabel("Value Range")
axes[1, 1].set_ylabel("Frequency")
plt.tight_layout()
plt.show()
```

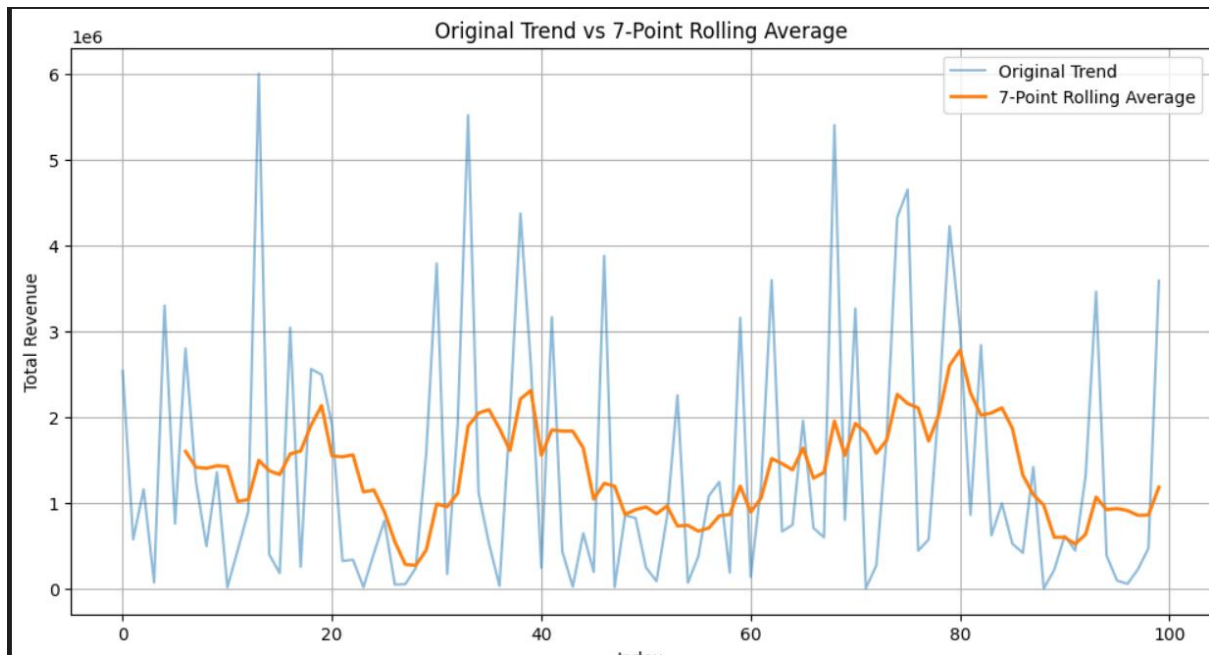



Question: - 10

Create a rolling average of window size 7 on a dataset and compare it with the original trend using Matplotlib.

Ans: Code & Output

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv(r"D:\Data Science with AI & ML\SITP-12th-May\PANDAS\sales.csv")
col = "Total Revenue"
df["Rolling_Avg_7"] = df[col].rolling(window=7).mean()
plt.figure(figsize=(12,6))
plt.plot(df.index, df[col], label="Original Trend", alpha=0.5)
plt.plot(df.index, df["Rolling_Avg_7"], label="7-Point Rolling Average", linewidth=2)
plt.title("Original Trend vs 7-Point Rolling Average")
plt.xlabel("Index")
plt.ylabel("Total Revenue")
plt.legend()
plt.grid(True)
plt.show()
```



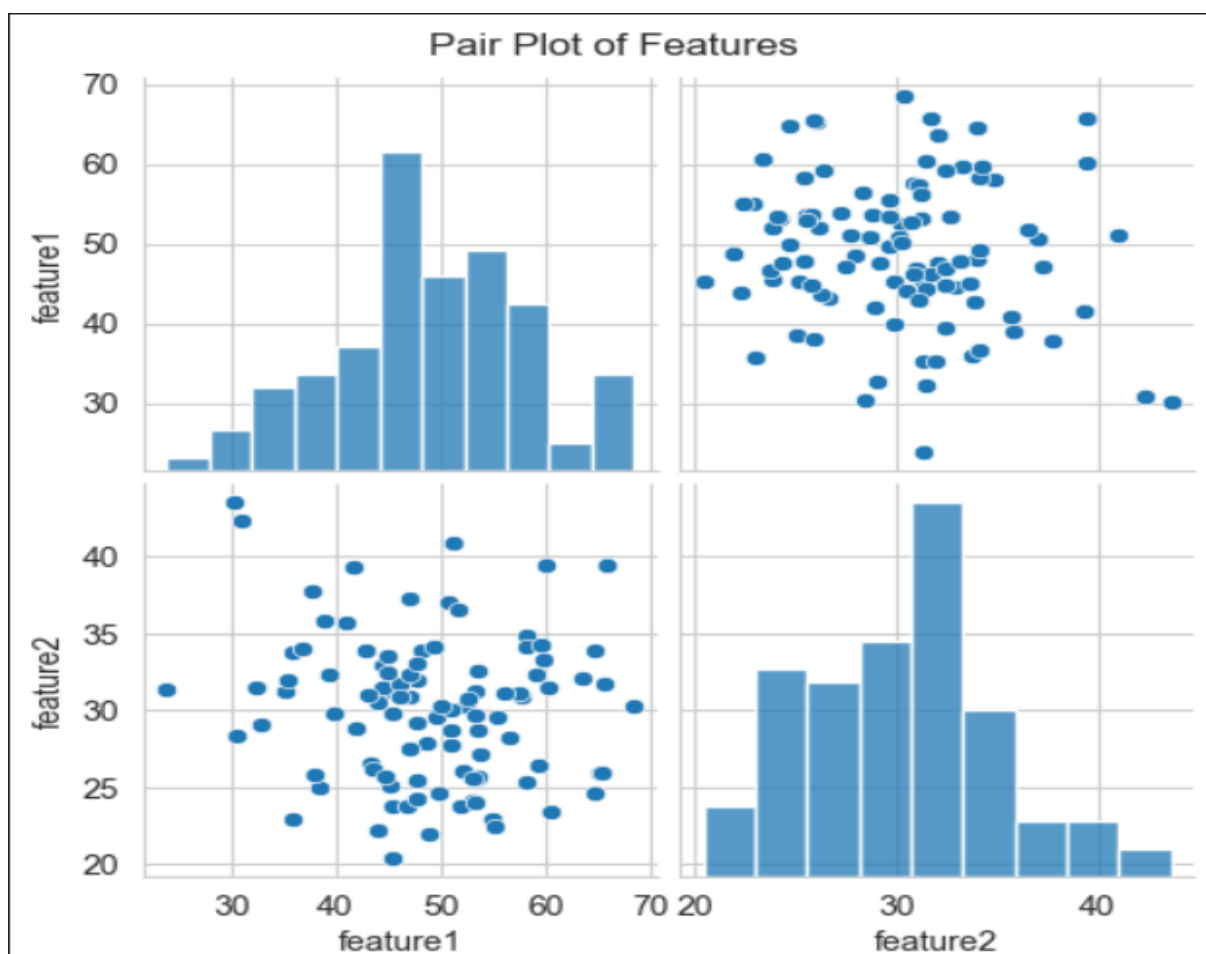
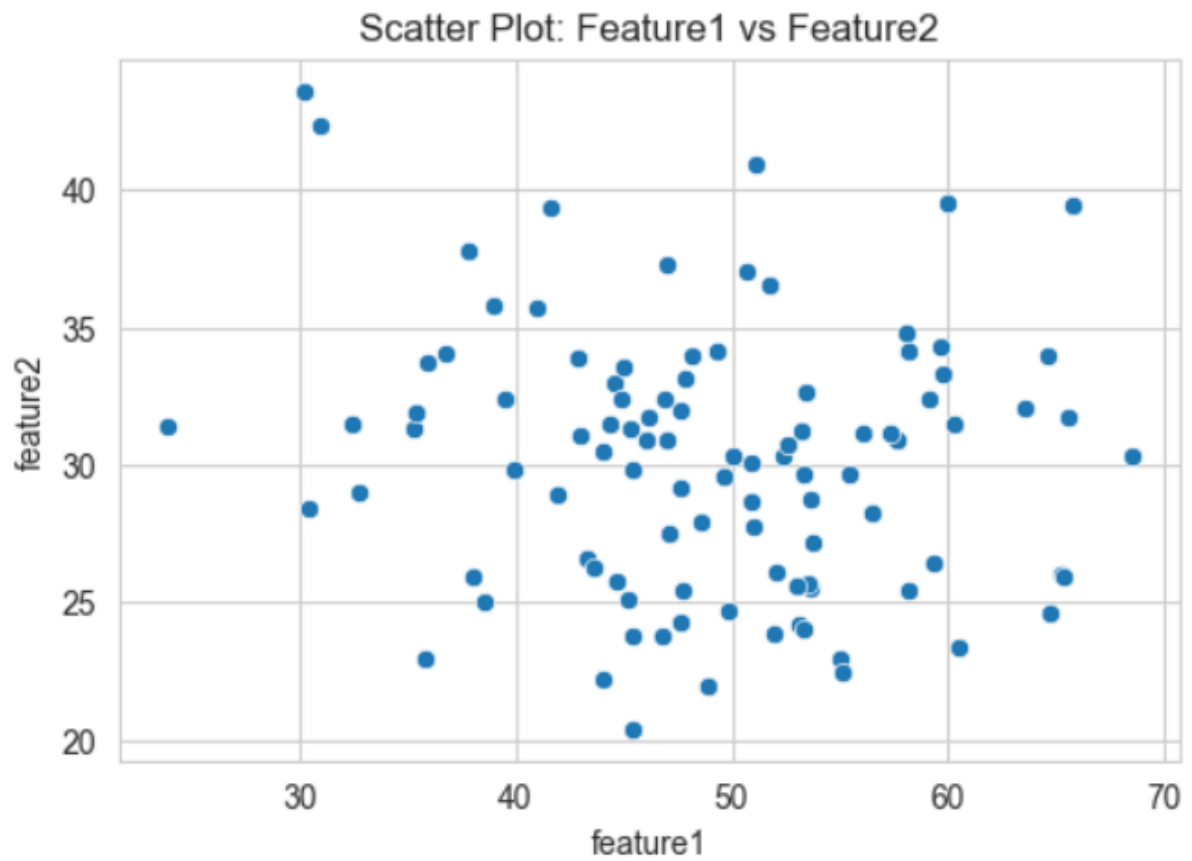
Question: - 11

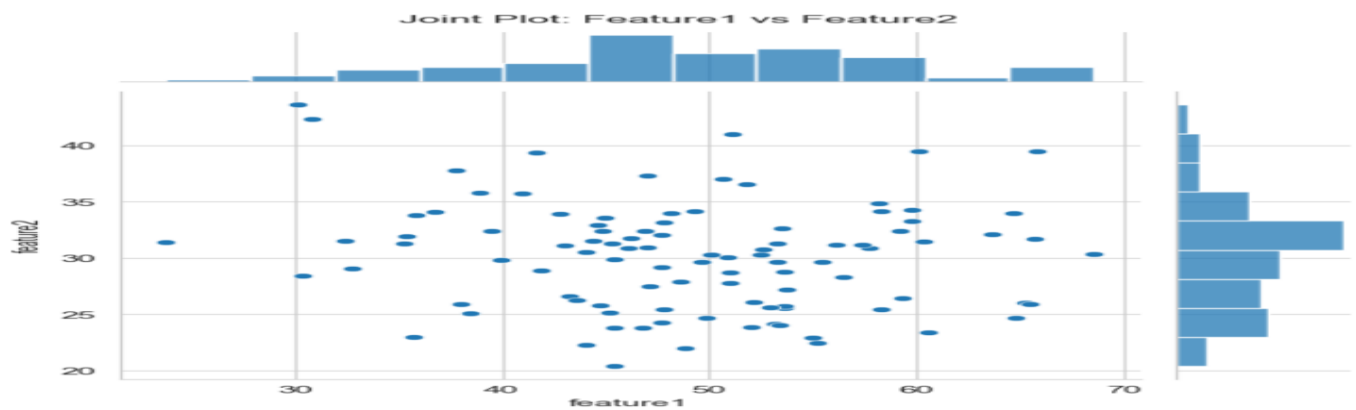
Using Seaborn, compare two numerical features using:

Scatter Plot, Pair Plot, Joint Plot

Ans: Code & Output

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
np.random.seed(42)
df = pd.DataFrame({
    "feature1": np.random.normal(50, 10, 100),
    "feature2": np.random.normal(30, 5, 100)
})
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x="feature1", y="feature2")
plt.title("Scatter Plot: Feature1 vs Feature2")
plt.show()
sns.pairplot(df)
plt.suptitle("Pair Plot of Features", y=1.02)
plt.show()
sns.jointplot(data=df, x="feature1", y="feature2", kind="scatter")
plt.suptitle("Joint Plot: Feature1 vs Feature2", y=1.02)
plt.show()
```





Question: - 12

Build a mini-EDA report that includes: Missing Values Analysis, Correlation Analysis, Outlier Detection, Feature Distribution Visualization

Ans: Code & Output

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv(r"D:\Data Science with AI & ML\SITP-12th-May\PANDAS\sales.csv")
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:\n", df.head())
# MISSING VALUES ANALYSIS
print("\n Missing Values:\n")
print(df.isnull().sum())
plt.figure(figsize=(10,4))
sns.heatmap(df.isnull(), cbar=False, cmap="viridis")
plt.title("Missing Values Heatmap")
plt.show()
# CORRELATION ANALYSIS
num_df = df.select_dtypes(include=["int64", "float64"])
corr = num_df.corr()
print("\n Correlation Matrix:\n", corr)
plt.figure(figsize=(10,6))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
# OUTLIER DETECTION
col = "Total Revenue"
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
print("\n Outlier Summary")
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Number of Outliers:", len(outliers))
plt.figure(figsize=(6,4))
sns.boxplot(x=df[col])
plt.title("Box Plot - Total Revenue Outliers")
plt.show()
# FEATURE DISTRIBUTION
num_cols = ["Units Sold", "Unit Price", "Total Revenue", "Total Profit"]
df[num_cols].hist(figsize=(12,8), bins=20)
plt.suptitle("Feature Distributions", fontsize=14)
plt.show()
# END OF EDA REPORT
print("\n Mini EDA Report Completed Successfully.")
```

First 5 rows:

	Region	Country	Item Type
0	Australia and Oceania	Tuvalu	Baby Food
1	Central America and the Caribbean	Grenada	Cereal
2	Europe	Russia	Office Supplies
3	Sub-Saharan Africa	Sao Tome and Principe	Fruits
4	Sub-Saharan Africa	Rwanda	Office Supplies

	Sales Channel	Order Priority	Order Date	Order ID	Ship Date	Units Sold
0	Offline	H	5/28/2010	669165933	6/27/2010	9925
1	Online	C	8/22/2012	963881480	9/15/2012	2804
2	Offline	L	5/2/2014	341417157	5/8/2014	1779
3	Online	C	6/20/2014	514321792	7/5/2014	8102
4	Offline	L	2/1/2013	115456712	2/6/2013	5062

	Unit Price	Unit Cost	Total Revenue	Total Cost	Total Profit
0	255.28	159.42	2533654.00	1582243.50	951410.50
1	205.70	117.11	576782.80	328376.44	248406.36
2	651.21	524.96	1158502.59	933903.84	224598.75
3	9.33	6.92	75591.66	56065.84	19525.82
4	651.21	524.96	3296425.02	2657347.52	639077.50

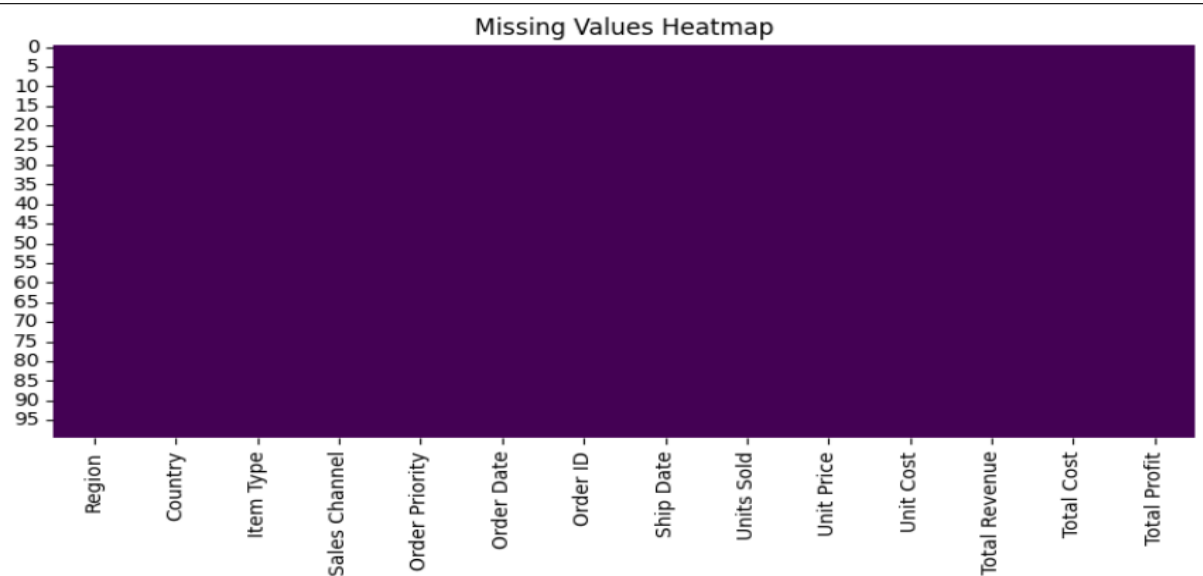
Missing Values:

Region	0
Country	0
Item Type	0
Sales Channel	0

```

Order Priority    0
Order Date       0
Order ID         0
Ship Date        0
Units Sold       0
Unit Price       0
Unit Cost        0
Total Revenue    0
Total Cost       0
Total Profit     0
dtype: int64

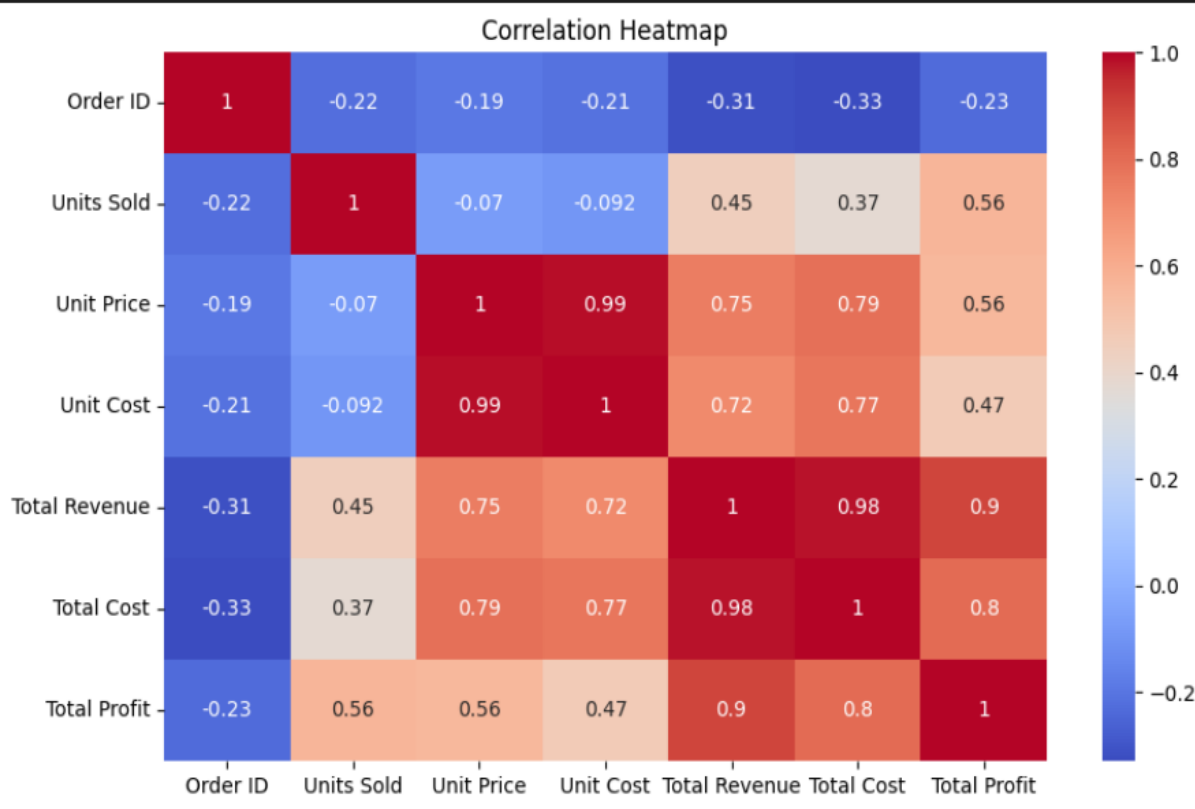
```



Correlation Matrix:

	Order ID	Units Sold	Unit Price	Unit Cost	Total Revenue
Order ID	1.000000	-0.222907	-0.190941	-0.213201	-0.314688
Units Sold	-0.222907	1.000000	-0.070486	-0.092232	0.447784
Unit Price	-0.190941	-0.070486	1.000000	0.987270	0.752360
Unit Cost	-0.213201	-0.092232	0.987270	1.000000	0.715623
Total Revenue	-0.314688	0.447784	0.752360	0.715623	1.000000
Total Cost	-0.328944	0.374746	0.787905	0.774895	0.983928
Total Profit	-0.234638	0.564550	0.557365	0.467214	0.897327

	Total Cost	Total Profit
Order ID	-0.328944	-0.234638
Units Sold	0.374746	0.564550
Unit Price	0.787905	0.557365
Unit Cost	0.774895	0.467214
Total Revenue	0.983928	0.897327
Total Cost	1.000000	0.804091
Total Profit	0.804091	1.000000

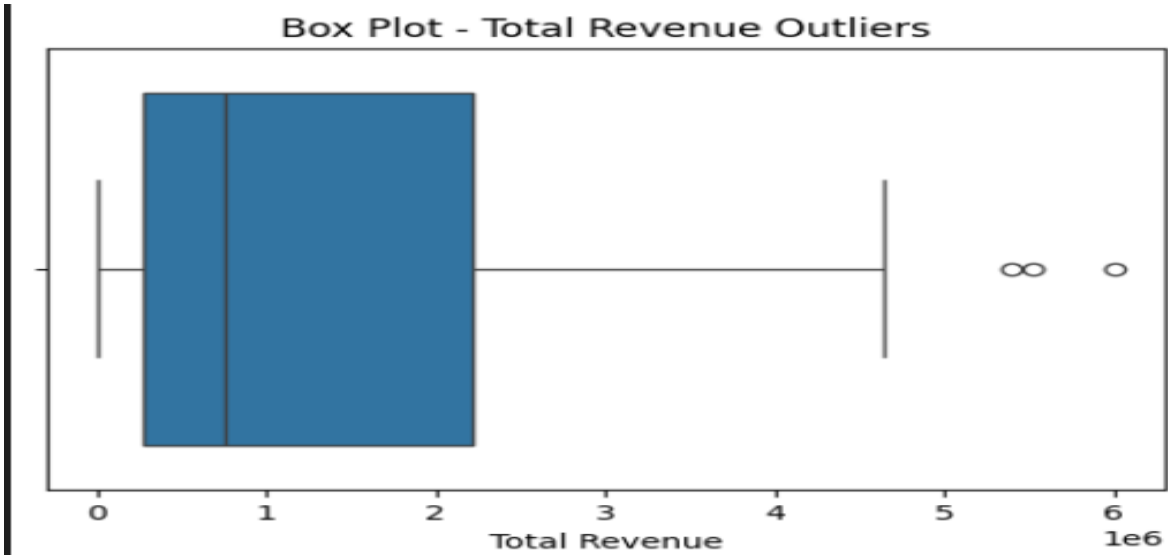


Outlier Summary

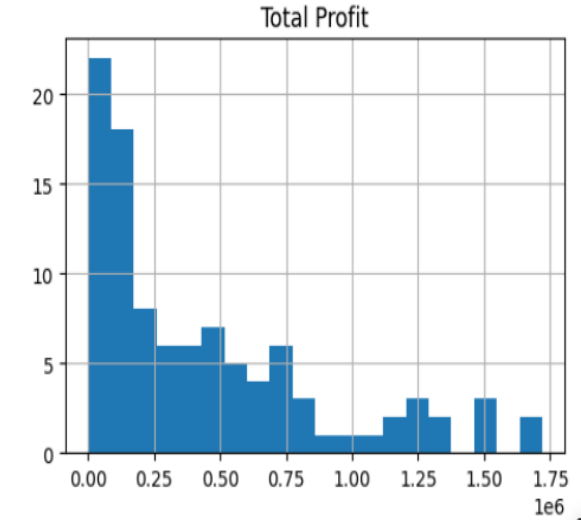
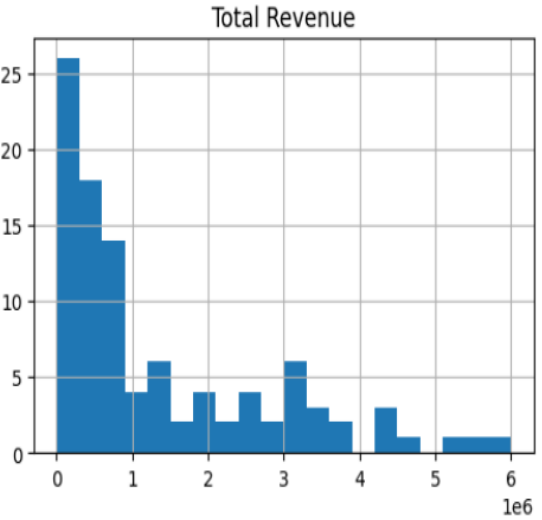
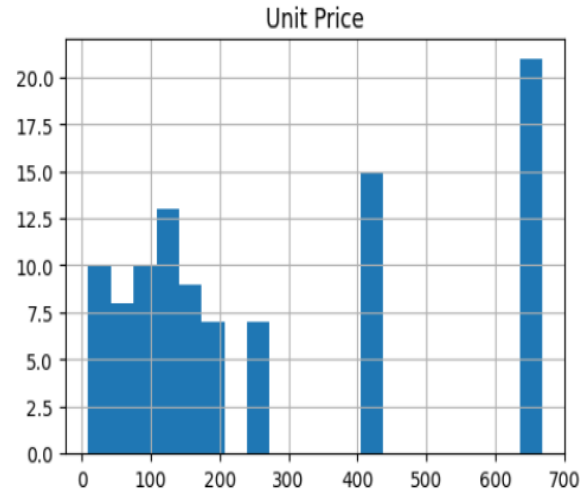
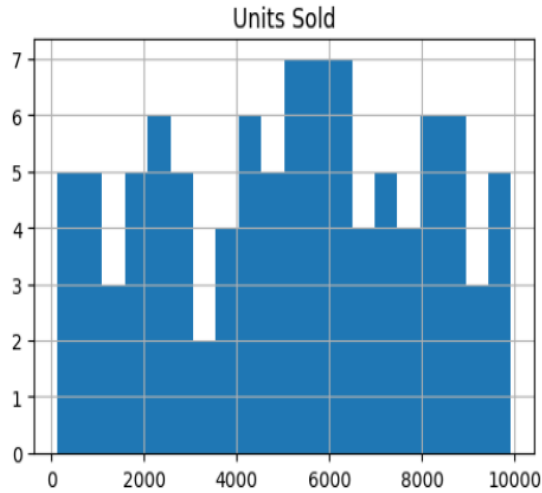
Lower Bound: -2646263.9925

Upper Bound: 5127029.8875

Number of Outliers: 3



Feature Distributions



Mini EDA Report Completed Successfully.